BANKON × 0G INFT (ERC-7857) Integration Guide — Definitive Technical Specification

TL;DR

- **Deploy `MindXAgentINFT` (ERC-7857) on 0G Aristotle Mainnet (chain ID 16661, RPC `https://evmrpc.0g.ai`, explorer `chainscan.0g.ai`)** using the 0G Foundation reference implementation at `github.com/0gfoundation/0g-agent-nft` (branch `eip-7857-draft`) as the canonical interface base — Galileo testnet (chain ID 16601) is for pre-flight only; the spec text and reference verifier are quoted verbatim from EIP-7857 at `eips.ethereum.org/EIPS/eip-7857`.
- **Settle agent usage in Algorand-native USDC (ASA ID 31566704, issued by Circle) over HTTP 402 via GoPlausible's `@x402-avm` packages** (`github.com/GoPlausible/x402-avm`, v2.6.1, Apache-2.0), exposed through the `parsec-wallet` branded abstraction; CAIP-2 identifiers are `algorand:wGHE2Pwdvd7S12BL5FaOP20EGYesN73ktiC1qzkkit8=` (mainnet) and `algorand:SGO1GKSzyE7IEPItTxCByw9x8FmnrCDexi9/cOUJOiI=` (testnet). GitHub GitHub
- **DAIO governance is dual-chain**: the Algorand constitutional layer holds the Boardroom/War Council charter (NFD V3-anchored identities, ARC-19 mutable metadata); the EVM economic layer (`DAIOAgentGovernor` on 0G Aristotle) runs the 13-Sun-Tzu verdict aggregator (≥10 WAGE, ≥8 SUBDUE, 5–7 HOLD, <5 WITHDRAW), bridged by openBDK and BONAFIDE attestations.

Key Findings

The 0G Foundation INFT stack is production-ready as of May 2026. Per Messari's commissioned project report *Understanding 0G: A Comprehensive Overview*, "0G Chain's Aristotle mainnet launched on Sept. 21, 2025, and the 0G token launched on Sept. 22, 2025" — that is, mainnet went live September 21, 2025 (chain ID 16661, native `0G` gas token), with the TGE following on September 22. The ERC-7857 standard ("AI Agents NFT with Private Metadata"), authored by Ming Wu (@sparkmiw), Jason Zeng (@zenghbo), Wei Wu (@Wilbert957), and Michael Heinrich (@michaelomg) and entered into the ethereum/ERCs repository via PR #824 with an initial commit dated January 3, 2025, defines three normative interfaces — `IERC7857`, `IERC7857Metadata`, `IERC7857DataVerifier` — that the canonical 0G Foundation reference implementation at `github.com/0gfoundation/0g-agent-nft` (CC0-1.0, last commit March 4, 2026) realises through `AgentNFT.sol`, `AgentNFTImpl.sol` behind an `AgentNFTBeacon`, and a `TEEVerifier.sol` oracle. The spec mandates a `TransferValidityProof` composed of an `AccessProof` (receiver-signed) and an `OwnershipProof` (TEE- or ZKP-attested), and the on-

chain `verifyTransferValidity` function must verify both before state transitions occur. Messari + 5

The Algorand x402 settlement rail is the GoPlausible implementation, published as the `@x402-avm/*` npm scope (core, avm, evm, express, next, hono, axios, extensions, paywall) at version 2.6.1 and as the `x402-avm` Python package on PyPI. Parsec (the user's branded layer) consumes this directly; there is no separate Algorand-side cryptography to re-implement. CAIP-2 identifiers, USDC ASA IDs (mainnet `31566704`, testnet `10458941`), atomic-group fee abstraction, and the `PAYMENT-SIGNATURE` header flow are all standardised. Coinbase's upstream x402 reference (`github.com/coinbase/x402`) hosts the canonical Algorand exact-scheme spec at `specs/schemes/exact/scheme_exact_algo.md`. GitHub GitHub

The BANKON ecosystem already has the right primitives in place. AllChain at `agenticplace.pythai.net/allchain.html` is the CAIP-2 loader; we add `eip155:16661` (0G Aristotle), `eip155:16601` (0G Galileo), `algorand:wGHE2Pwdvd7S12BL5FaOP20EGYesN73ktiC1qzkkit8=` (Algorand mainnet), `eip155:5042002` (Arc Testnet), and the standard `eip155:1/8453/137` for Ethereum/Base/Polygon. NeuralNode at `0x024b464ec595F20040002237680026bf006e8F90` on Polygon and the Unified Bridge at `0x2a3DD3EB832aF982ec71669E178424b10Dca2EDe` on Ethereum mainnet are the existing L1 anchors; THRUST BSC (`0x969F60Bfe17962E0f061B434596545C7b6Cd6Fc4`) and Chainlog (`0xdA0Ab1e0017DEbCd72Be8599041a2aa3bA7e740F`) remain the BSC and registry endpoints; Base USDC at `0x833589fCD6eDb6E08f4c7C32D4f71b54bdA02913` is the EVM-side stablecoin counterpart to Algorand ASA `31566704`. Go Packages

Details

1. ERC-7857 — Canonical Specification

The EIP-7857 draft at `https://eips.ethereum.org/EIPS/eip-7857` (markdown source: `https://github.com/ethereum/EIPs/blob/master/EIPS/eip-7857.md`; the equivalent ERCs-repo PR is `https://github.com/ethereum/ERCs/pull/824`) defines a standalone NFT interface that intentionally does *not* inherit from ERC-721, although implementations MAY implement ERC-721 for marketplace compatibility. The interface set comprises: Ethereum GitHub

```solidity
```

```
// SPDX-License-Identifier: Apache-2.0
// (c) 2026 BANKON — all rights reserved.
pragma solidity ^0.8.24;

enum OracleType { TEE, ZKP }

struct AccessProof {
    bytes32 oldDataHash;
    bytes32 newDataHash;
    bytes   nonce;
    bytes   encryptedPubKey;
    bytes   proof;
}

struct OwnershipProof {
    OracleType oracleType;
    bytes32 oldDataHash;
    bytes32 newDataHash;
    bytes   sealedKey;
    bytes   encryptedPubKey;
    bytes   nonce;
    bytes   proof;
}

struct TransferValidityProof {
    AccessProof    accessProof;
    OwnershipProof ownershipProof;
}

struct TransferValidityProofOutput {
    bytes32 oldDataHash;
    bytes32 newDataHash;
    bytes   sealedKey;
    bytes   encryptedPubKey;
    bytes   wantedKey;
    address accessAssistant;
    bytes   accessProofNonce;
    bytes   ownershipProofNonce;
}

interface IERC7857DataVerifier {
    function verifyTransferValidity(
        TransferValidityProof[] calldata _proofs
```

```
    ) external returns (TransferValidityProofOutput[] memory);
}

struct IntelligentData {
    string  dataDescription;
    bytes32 dataHash;
}

interface IERC7857Metadata {
    function name() external view returns (string memory);
    function symbol() external view returns (string memory);
    function intelligentDataOf(uint256 _tokenId)
        external view returns (IntelligentData[] memory);
}

interface IERC7857 {
    event Transferred(uint256 _tokenId, address indexed _from, address indexed _to);
    event Cloned(uint256 indexed _tokenId, uint256 indexed _newTokenId, address _fr
    event PublishedSealedKey(address indexed _to, uint256 indexed _tokenId, bytes[]
    event Authorization(address indexed _from, address indexed _to, uint256 indexed
    event AuthorizationRevoked(address indexed _from, address indexed _to, uint256 i
    event DelegateAccess(address indexed _user, address indexed _assistant);

    function verifier() external view returns (IERC7857DataVerifier);

    function iTransfer(address _to, uint256 _tokenId, TransferValidityProof[] callda
    function iClone(address _to, uint256 _tokenId, TransferValidityProof[] calldata
        external returns (uint256 _newTokenId);

    function authorizeUsage(uint256 _tokenId, address _user) external;
    function revokeAuthorization(uint256 _tokenId, address _user) external;

    function approve(address _to, uint256 _tokenId) external;
    function setApprovalForAll(address _operator, bool _approved) external;
    function delegateAccess(address _assistant) external;

    function ownerOf(uint256 _tokenId) external view returns (address);
    function authorizedUsersOf(uint256 _tokenId) external view returns (address[] me
    function getApproved(uint256 _tokenId) external view returns (address);
    function isApprovedForAll(address _owner, address _operator) external view retur
    function getDelegateAccess(address _user) external view returns (address);
}
```

The sealed-metadata transfer flow is normative and proceeds in five steps: encryption and commitment (the agent's model/memory/character is encrypted and `keccak256`-committed); transfer initiation (a TEE or ZKP oracle decrypts the old ciphertext, generates a fresh symmetric key, and re-encrypts); receiver sealing (the new key is encrypted under the receiver's secp256k1 public key into a `sealedKey`); on-chain verification (`verifyTransferValidity` checks both `AccessProof` — a signature by the receiver over `(oldDataHash, newDataHash, encryptedPubKey, nonce)` — and `OwnershipProof` — a TEE attestation that the re-encryption was performed correctly); and finalisation (`iTransfer` updates owner state and emits `PublishedSealedKey` so the receiver can decrypt off-chain). GitHub GitHub

The reference Verifier contract at `https://github.com/0gfoundation/0g-agent-nft/blob/main/contracts/TEEVerifier.sol` (the eip-7857-draft branch is at `https://github.com/0glabs/0g-agent-nft/tree/eip-7857-draft`) implements `BaseVerifier` with a `usedProofs` replay-attack mapping, a 7-day `maxProofAge`, and an upgradeable `AccessControlUpgradeable + PausableUpgradeable` pattern. The TEE attestation is recovered with OpenZeppelin's ECDSA library against the Ethereum signed-message digest of `(oldDataHash, newDataHash, sealedKey, encryptedPubKey, nonce)`. The ZKP branch is reserved in the standard but not yet implemented in the 0G reference (the spec notes "TODO: add ZKP verification"). GitHub

2. 0G Network Infrastructure

The 0G stack comprises four production components, all live on Aristotle mainnet:

0G Chain — Aristotle mainnet at chain ID 16661, RPC `https://evmrpc.0g.ai`, explorer `https://chainscan.0g.ai`, native token `0G`. Galileo testnet V3 at chain ID 16601 (per the deepnewz announcement and the awesome-0g registry; note that ChainList still lists 16602 as an alternate Galileo entry, and the docs.0g.ai testnet page also shows 16602 in some Hardhat snippets — this is a known docs inconsistency, treat 16601 as the live value confirmed by the official thirdweb chain page at `thirdweb.com/0g-galileo-testnet-16601`). Cosmos+Ethermint stack at `github.com/0glabs/0g-chain`. Foundry deployment requires `evm_version = "cancun"` per the official docs. GitHub + 3

0G Storage — Production contract addresses on Aristotle mainnet (verified from `https://docs.0g.ai/developer-hub/mainnet/mainnet-overview`): Flow at `0x62D4144dB0F0a6fBBaeb6296c785C71B3D57C526`, Mine at `0xCd01c5Cd953971CE4C2c9bFb95610236a7F414fe`, Reward at `0x457aC76B58ffcDc118AABD6DbC63ff9072880870`. Storage indexer at `https://indexer-storage-turbo.0g.ai`. Galileo testnet equivalents (per `https://docs.0g.ai/developer-hub/testnet/testnet-overview`): Flow `0x22E03a6A89B950F1c82ec5e74F8eCa321a105296`, Mine `0x00A9E9604b0538e06b268Fb297Df333337f9593b`, Reward `0xA97B57b4BdFEA2D0a25e535bd849ad4e6C440A69`, DAEntrance `0xE75A073dA5bb7b0eC622170Fd268f35E675a957B` (the mainnet docs page does not currently list

a separate `DAEntrance` — DA appears to be served through Flow at Aristotle v1.0.4). Go SDK at `https://github.com/0glabs/0g-storage-client` with packages `core` (Merkle tree + AES-256-CTR client-side encryption), `node` (`ZgsClient.DownloadSegmentWithProof`), `kv` (stream KV with `UploadOption.EncryptionKey`), and `transfer`. TypeScript SDK at `npm install @0glabs/0g-ts-sdk`. Contracts at `https://github.com/0glabs/0g-storage-contracts`. `0g + 4`

0G Compute — Marketplace SDK at `https://github.com/0glabs/0g-serving-broker` (npm `@0glabs/0g-serving-broker`). The broker exposes a two-tier account system (main account funded from wallet, per-provider sub-accounts with 24-hour refund lock) and OpenAI-compatible inference endpoints. Service registration returns `ServiceStructOutput { provider, serviceType, url, inputPrice, outputPrice, model, verifiability }`, where `verifiability == "TeeML"` indicates TEE-attested inference. The user-broker repository is `https://github.com/0gfoundation/0g-serving-user-broker`. Compute starter kit at `0gfoundation/0g-compute-ts-starter-kit`.

0G DA (Data Availability) — Used for agent state checkpointing. Galileo `DAEntrance` at `0xE75A073dA5bb7b0eC622170Fd268f35E675a957B`. Stream IDs in the storage layer also serve DA-style append-only logs; the BANKON pattern is to checkpoint mindX agent state hashes at every CONCLAVE consensus tick.

3. Production Solidity Contracts

The Foundry workspace follows the cypherpunk2048 flat snake_case standard:

```
mindx_agent_inft/
├── foundry.toml
├── remappings.txt
├── src/
│   ├── mindx_agent_inft.sol          # MindXAgentINFT contract
│   ├── daio_agent_governor.sol       # DAIOAgentGovernor contract
│   ├── mindx_x402_middleware.sol     # MindXX402Middleware contract
│   ├── interfaces/
│   │   ├── i_erc7857.sol
│   │   ├── i_erc7857_metadata.sol
│   │   └── i_erc7857_data_verifier.sol
│   └── verifiers/
│       └── tee_verifier_adapter.sol  # BONAFIDE-bridged TEE adapter
├── script/
│   └── deploy.s.sol
└── test/
    ├── mindx_agent_inft.t.sol
    ├── daio_agent_governor.t.sol
    └── mindx_x402_middleware.t.sol
```

`src/mindx_agent_inft.sol` wraps the 0G `AgentNFT` reference with BONAFIDE attestation gating. The contract is deployed behind an upgradeable beacon (matching the 0G deployment pattern's `AgentNFTBeacon`) and exposes `mintAgent(address to, IntelligentData[] calldata data, OwnershipProof[] calldata proofs)`, `transferAgent` (aliased to `iTransfer`), `cloneAgent` (aliased to `iClone`), and `authorizeUsage`. Every mint requires a signed BONAFIDE attestation from `Senatus` (the BONAFIDE governance contract); every transfer additionally requires the receiver to hold a non-revoked `Fides` reputation token. The contract stores 0G Storage URIs (the `dataHash` of `IntelligentData` is the Merkle root returned by `0g-storage-client` upload) and emits a `Listed(uint256 tokenId, uint256 priceUsdcAlgo)` event that AgenticPlace indexes for marketplace display:

```solidity
```

```solidity
// SPDX-License-Identifier: Apache-2.0
// (c) 2026 BANKON — all rights reserved.
pragma solidity ^0.8.24;

import {IERC7857, TransferValidityProof, OwnershipProof, IntelligentData}
    from "./interfaces/i_erc7857.sol";
import {IERC7857Metadata} from "./interfaces/i_erc7857_metadata.sol";
import {IERC7857DataVerifier} from "./interfaces/i_erc7857_data_verifier.sol";

interface ISenatus { function attest(address subject, bytes32 claim) external view r
interface IFides   { function reputationOf(address subject) external view returns (u

contract MindXAgentINFT is IERC7857, IERC7857Metadata {
    string  public override name;
    string  public override symbol;
    IERC7857DataVerifier public override verifier;
    ISenatus public senatus;
    IFides   public fides;
    address  public agenticPlace;

    struct TokenData {
        address owner;
        address[] authorizedUsers;
        address approvedUser;
        IntelligentData[] iDatas;
        string  zeroGStorageRoot; // 0G Storage merkle root URI
    }

    mapping(uint256 => TokenData)                          internal tokens;
    mapping(address => mapping(address => bool))           internal operatorApprov
    mapping(address => address)                            internal accessAssistan
    uint256 internal nextTokenId;

    event Listed(uint256 indexed tokenId, address indexed seller, uint256 priceUsdcM
    event Minted(uint256 indexed tokenId, address indexed creator, address indexed c

    constructor(
        string memory _n, string memory _s,
        IERC7857DataVerifier _v, ISenatus _sen, IFides _fid, address _market
    ) {
        name = _n; symbol = _s;
        verifier = _v; senatus = _sen; fides = _fid; agenticPlace = _market;
        nextTokenId = 1;
```

```
}

function mintAgent(
    address to,
    IntelligentData[] calldata data,
    string calldata zeroGRoot,
    bytes32 attestClaim
) external returns (uint256 tokenId) {
    require(senatus.attest(msg.sender, attestClaim), "BONAFIDE: not attested");
    tokenId = nextTokenId++;
    TokenData storage t = tokens[tokenId];
    t.owner = to;
    t.zeroGStorageRoot = zeroGRoot;
    for (uint256 i; i < data.length; ++i) t.iDatas.push(data[i]);
    emit Minted(tokenId, msg.sender, to, data[0].dataHash);
}

function iTransfer(
    address to,
    uint256 tokenId,
    TransferValidityProof[] calldata proofs
) external override {
    TokenData storage t = tokens[tokenId];
    require(t.owner == msg.sender || t.approvedUser == msg.sender, "not authoriz
    require(fides.reputationOf(to) > 0, "BONAFIDE: receiver lacks Fides");
    TransferValidityProofOutput[] memory outs = verifier.verifyTransferValidity(
    for (uint256 i; i < outs.length; ++i) {
        require(outs[i].oldDataHash == t.iDatas[i].dataHash, "hash mismatch");
        t.iDatas[i].dataHash = outs[i].newDataHash;
        bytes[] memory keys = new bytes[](1);
        keys[0] = outs[i].sealedKey;
        emit PublishedSealedKey(to, tokenId, keys);
    }
    t.owner = to;
    emit Transferred(tokenId, msg.sender, to);
}

function iClone(
    address to,
    uint256 tokenId,
    TransferValidityProof[] calldata proofs
) external override returns (uint256 newTokenId) {
    TokenData storage src = tokens[tokenId];
    require(src.owner == msg.sender, "not owner");
```

```
        TransferValidityProofOutput[] memory outs = verifier.verifyTransferValidity(
        newTokenId = nextTokenId++;
        TokenData storage dst = tokens[newTokenId];
        dst.owner = to;
        dst.zeroGStorageRoot = src.zeroGStorageRoot;
        for (uint256 i; i < outs.length; ++i) {
            dst.iDatas.push(IntelligentData({
                dataDescription: src.iDatas[i].dataDescription,
                dataHash:        outs[i].newDataHash
            }));
        }
        emit Cloned(tokenId, newTokenId, msg.sender, to);
    }

    function authorizeUsage(uint256 tokenId, address user) external override {
        require(tokens[tokenId].owner == msg.sender, "not owner");
        tokens[tokenId].authorizedUsers.push(user);
        emit Authorization(msg.sender, user, tokenId);
    }
    // ... revokeAuthorization, approve, setApprovalForAll, delegateAccess, view met
}
```

src/daio_agent_governor.sol implements the dual-chain DAIO. The EVM-side governor on 0G Aristotle holds a 13-bit verdict bitmap (one bit per Sun Tzu chapter assessor) and an enum `Verdict { WAGE, SUBDUE, HOLD, WITHDRAW }`. The aggregation rule is enforced on-chain: ≥10 set bits → WAGE, ≥8 → SUBDUE, 5–7 → HOLD, <5 → WITHDRAW. CEO + 7 Soldier roster addresses are stored in an `EnumerableSet`, and a `CONCLAVE` epoch counter increments per agreed verdict. The Algorand constitutional layer (BOARDROOM/WARCOUNCIL charter) is referenced by an `bytes32 algorandCharterHash` that points to an NFD V3 record under `bankon.algo`; openBDK relays state updates by submitting Ed25519 multisig proofs that this contract verifies via a precompile-style adapter.

src/mindx_x402_middleware.sol is a thin on-chain receipt registry: the actual 402 payment flow happens off-chain via Algorand atomic transaction groups (per the GoPlausible scheme), but this contract logs `Receipt(tokenId, payer, asaId=31566704, amountMicro, algoTxId)` so that mindX inference can be EVM-verifiable. The contract trusts a signer-set `Censura` oracle (a BONAFIDE-attested facilitator address) to attest payment finality.

All contracts compile under Foundry with `solc 0.8.24` and `evm_version = "cancun"` (mandated by 0G docs). OG Labs

Parsec is the BANKON-branded TypeScript abstraction over GoPlausible's `@x402-avm/avm` and `@x402-avm/core` packages — confirmed real and active at `github.com/GoPlausible/x402-avm` v2.6.1 (Apache-2.0). No fork is necessary; Parsec is a façade pattern that re-exports types and provides BANKON-specific defaults (network presets pinned to Algorand mainnet `algorand:wGHE2Pwdvd7S12BL5FaOP20EGYesN73ktiC1qzkkit8=`, USDC ASA `31566704`, the BANKON facilitator URL, and BONAFIDE attestation hooks). npm

The Parsec front-end SDK pattern from AgenticPlace:

typescript

```typescript
// agenticplace/src/lib/parsec.ts
import { x402Client, HTTPFacilitatorClient } from "@x402-avm/core";
import { registerExactAvmClient } from "@x402-avm/avm/exact/client";
import { ALGORAND_MAINNET_CAIP2, USDC_MAINNET_ASA_ID } from "@x402-avm/avm";
import { PeraWalletConnect } from "@perawallet/connect";

export async function parsecPay({ url, signer }: { url: string; signer: any }) {
  const client = new x402Client({
    facilitator: new HTTPFacilitatorClient({ url: "https://parsec.bankon.eth.link/fa
  });
  registerExactAvmClient(client, signer);
  return client.fetch(url); // intercepts 402, builds atomic group, retries with PAY
}
```

The wallet stack uses ARC-52 HD derivation (`github.com/algorandfoundation/xHD-Wallet-API-ts`, BIP32-Ed25519 over non-linear keyspace at path `m/44'/283'/account'/change/index`), Pera-compatible multisig via `@perawallet/connect` (`github.com/perawallet/connect`), and Algorand state proofs for quantum-aware finality (the State Proof transactions are already produced by go-algorand validators and verifiable off-chain). The `xHD-Wallet-API-ts` library exposes Peikert-mode derivation by default (per its README: "Ammendment to the standard mode to allow for a more secure derivation of keys by giving more entropy to zL"). GitHub + 2

5. AllChain — CAIP-2 Chain Map

The `agenticplace.pythai.net/allchain.html` CAIP-2 loader registers the following chain identifiers as of May 2026:

Network	CAIP-2	Chain ID	Role
0G Aristotle Mainnet	`eip155:16661`	16661	INFT prim deployme
0G Galileo Testnet V3	`eip155:16601`	16601	Pre-flight only
Ethereum Mainnet	`eip155:1`	1	Unified Bridge anchor
Base Mainnet	`eip155:8453`	8453	EVM USD settlement
Polygon Mainnet	`eip155:137`	137	NeuralNo
BSC	`eip155:56`	56	THRUST
Algorand Mainnet	`algorand:wGHE2Pwdvd7S12BL5FaOP20EGYesN73ktiC1qzkkit8=`	n/a	x402 / NFI DAIO constitutic
Algorand Testnet	`algorand:SGO1GKSzyE7IEPItTxCByw9x8FmnrCDexi9/cOUJOiI=`	n/a	x402 pre-flight
Arc Testnet (Circle)	`eip155:5042002`	5042002	USDC mir settlement

Asset registry: USDC on Algorand mainnet is ASA `31566704`, issued by Circle (Circle acquired Centre Consortium in August 2023 and became the sole USDC issuer; per circle.com/multi-chain-usdc the official entry reads "USDC on Algorand — Token Standard: ASA — Mainnet Address: 31566704"). The ASA has 6 decimals; circulating supply on Algorand stands at approximately $62.4M USDC as of May 2026 per usdc.cool/algorand. The 18.4 trillion "total supply" figure visible on Pera Explorer is the max-uint64 minting ceiling, not the actual circulating supply. USDC on Base mainnet is `0x833589fCD6eDb6E08f4c7C32D4f71b54bdA02913`.

NeuralNode is `0x024b464ec595F20040002237680026bf006e8F90` on Polygon, Unified Bridge `0x2a3DD3EB832aF982ec71669E178424b10Dca2EDe` on Ethereum mainnet, THRUST `0x969F60Bfe17962E0f061B434596545C7b6Cd6Fc4` on BSC, Chainlog `0xdA0Ab1e0017DEbCd72Be8599041a2aa3bA7e740F`. Circle + 2

6. mindX API Integration

The mindX API at `mindx.pythai.net` is gated by INFT ownership. The flow:

1. A user mints `MindXAgentINFT` token #N via AgenticPlace, supplying agent character JSON + model weight reference. The character document is encrypted client-side with AES-256-CTR (using `0g-storage-client`'s `UploadOption.EncryptionKey`), uploaded to 0G Storage, and the returned Merkle root becomes `IntelligentData.dataHash`. GitHub
2. The mint emits `Minted(tokenId, creator, owner, metaRoot)`. A mindX provisioning daemon (subscribed to chain events via `ethers.WebSocketProvider`) downloads the encrypted blob, decrypts it inside a TEE-attested 0G Compute container, and instantiates the agent's Soul/Mind/Hands cognition stack.
3. API requests to `mindx.pythai.net/v1/agents/{tokenId}/chat` carry an EIP-712 ownership signature in the `X-INFT-Owner` header; the gateway calls `MindXAgentINFT.ownerOf(tokenId)` and `MindXAgentINFT.authorizedUsersOf(tokenId)` to authorise. Unauthorised callers receive HTTP 402 with a Parsec payment requirement.
4. Inference uses the AI SDK v6 (`ai-sdk.dev`) `ToolLoopAgent` with the framework default `stopWhen: stepCountIs(20)` — per Vercel's AI SDK 6 launch post, "The ToolLoopAgent class provides a production-ready implementation that handles the complete tool execution loop…for up to 20 steps by default: stopWhen: stepCountIs(20)" — BONAFIDE attestation tools injected via `createMCPClient` (MCP transport over streamable HTTP), and the `@x402-avm/express` middleware enforcing the per-call USDC price (`$0.001`–`$0.10` depending on model class). `generateText` is used for synchronous calls; `streamText` for chat UIs. Vercel

typescript

```
// mindx.pythai.net/server/agent_runtime.ts
import { ToolLoopAgent, stepCountIs } from "ai";
import { createMCPClient } from "ai";
import { Senatus } from "@bankon/bonafide-tools";

const agent = new ToolLoopAgent({
  model: ogCompute("qwen/qwen-2.5-7b-instruct"),
  instructions: characterDoc.system,
  tools: { ...await mcpClient.tools(), ...Senatus.tools() },
  stopWhen: stepCountIs(20),
});
```

7. Foundry Test Suite

`foundry.toml` pins `solc = "0.8.24"`, `evm_version = "cancun"`, `optimizer_runs = 200`, and declares RPC endpoints for `og_mainnet = "https://evmrpc.0g.ai"`, `og_testnet = "https://evmrpc-testnet.0g.ai"`, `ethereum = "${ETH_RPC_URL}"`, `base = "https://mainnet.base.org"`, `polygon = "${POLYGON_RPC_URL}"`. Fork tests use `vm.createSelectFork("og_mainnet")` and exercise the live 0G Storage Flow contract `0x62D4144dB0F0a6fBBaeb6296c785C71B3D57C526` to verify Merkle root submission. Algorand state cannot be forked directly under EVM Foundry; instead, the test suite uses Wormhole-bridged Algorand state proofs (queried via the Wormhole guardian RPC) and stubs the rest via `vm.mockCall`.

Property-based fuzz tests over `iTransfer`:

solidity

```
function testFuzz_iTransfer_PreservesHashes(
    bytes32 oldHash, bytes32 newHash, bytes calldata sealedKey
) public {
    vm.assume(oldHash != bytes32(0) && newHash != bytes32(0) && oldHash != newHash);
    // ... build TransferValidityProof, mock verifier, assert post-state equals (new
}
```

forge-std (`forge-std/Test.sol`, `forge-std/console.sol`) and ds-test (`ds-test/test.sol`) are used throughout. Coverage target: 95% line, 90% branch.

8. Mainnet Deployment

`script/deploy.s.sol` deploys in this order: `TEEVerifierAdapter` → `MindXAgentINFT` (behind a beacon proxy) → `DAIOAgentGovernor` → `MindXX402Middleware`, each verified on

`chainscan.0g.ai` via the Hardhat-Etherscan v2 plugin pointed at `https://chainscan.0g.ai/open/api`. The multi-chain matrix: `0G Labs`

- **0G Aristotle (16661)** — `MindXAgentINFT`, `DAIOAgentGovernor`, `MindXX402Middleware`, `TEEVerifierAdapter`.
- **Polygon (137)** — L1Escrow integration that locks USDC against minted INFT IDs; reads `NeuralNode` at `0x024b464ec595F20040002237680026bf006e8F90` for reputation oracle.
- **Base (8453)** — EVM-side USDC settlement using USDC `0x833589fCD6eDb6E08f4c7C32D4f71b54bdA02913`; mirrored to Algorand via the bridge pattern.
- **Ethereum mainnet (1)** — Unified Bridge `0x2a3DD3EB832aF982ec71669E178424b10Dca2EDe` registers the agent NFT collection address for cross-chain proof.
- **Algorand mainnet** — NFD V3 records under `bankon.algo` for each agent, minted via `@txnlab/nfd-sdk` (`github.com/TxnLab/nfd-sdk`); ARC-19 ASA pairs minted per INFT for marketplace discoverability (`asa-list.tinyman.org` indexing).
- **BSC (56)** — THRUST `0x969F60Bfe17962E0f061B434596545C7b6Cd6Fc4` remains the BSC settlement endpoint; Chainlog `0xdA0Ab1e0017DEbCd72Be8599041a2aa3bA7e740F` is the registry of record.

Deployment command:

```bash
forge script script/deploy.s.sol:Deploy \
  --rpc-url $OG_MAINNET_RPC \
  --private-key $DEPLOYER_KEY \
  --evm-version cancun \
  --broadcast --verify \
  --verifier-url https://chainscan.0g.ai/open/api \
  --etherscan-api-key $OG_SCAN_KEY
```

9. AgenticPlace Marketplace

Frontend listing flow at `agenticplace.pythai.net`:

1. Owner navigates to *List Agent* and selects an INFT token from `MindXAgentINFT.ownerOf` filtered by their EOA.
2. The form requests a price (denominated in USDC-on-Algorand microunits) and a Parsec payTo Algorand address.
3. On submit, the page calls `MindXAgentINFT.approve(agenticPlaceContract, tokenId)` and emits a `Listed` event consumed by the marketplace indexer.

4. In parallel, the page mints a paired ARC-19 NFT on Algorand using `@txnlab/nfd-sdk` with the INFT `dataHash` as the IPFS reserve-address pointer, providing Algorand-native discoverability.
5. Buyers hit the listing page; the page issues HTTP 402 via `@x402-avm/next` (`paymentProxyFromConfig`), Parsec assembles the atomic group, and the GoPlausible facilitator settles to the seller's address. The marketplace indexer observes the `MindXX402Middleware.Receipt` event and triggers `iTransfer` via a meta-tx relayer once the receiver has co-signed the `AccessProof`.

The AllChain `allchain.html` CAIP-2 loader supplies network metadata to the marketplace UI; the same loader is reused by Parsec to pick the right RPC endpoint and explorer URL per chain.

10. DAIO Blockchain Deployment — BOARDROOM, War Council, CONCLAVE

The remaining DAIO work splits into three on-chain artifacts:

Boardroom (BOARDROOM.md) is encoded as `DAIOAgentGovernor.boardroomCharterHash` on 0G Aristotle, with the canonical document hash equal to the SHA-256 of the markdown source. The Algorand-side constitutional record is an NFD V3 entry under `boardroom.bankon.algo` whose ARC-19 reserve points to an IPFS CID of the same file. CEO + 7 Soldier roster addresses are seeded in a constructor argument and rotated only via a 2-of-3 multisig of CEO + 2 Soldiers (matching the `Curia` quorum rule from the BONAFIDE protocol).

War Council (WARCOUNCIL.md) maps the 13 Sun Tzu chapters (Calculations, Waging War, Attack by Stratagem, Tactical Dispositions, Energy, Weak Points and Strong, Manoeuvring, Variations, The Army on the March, Terrain, The Nine Situations, The Attack by Fire, The Use of Spies) to 13 assessor agent EOAs. Each assessor signs an EIP-712 `Verdict(epoch, chapter, outcome)` struct. `DAIOAgentGovernor.aggregateVerdict(uint256 epoch, bytes[] calldata sigs)` recovers all 13 signatures, counts the set bits, and emits `VerdictResolved(epoch, Verdict outcome)` where the enum cutoffs are exactly ≥10/13 → `WAGE`, ≥8/13 → `SUBDUE`, 5–7/13 → `HOLD`, <5/13 → `WITHDRAW`.

CONCLAVE coordination is implemented as a per-epoch event log: the CEO agent submits a `Convene(epoch)` transaction, each of the 7 Soldier agents responds with a `Vote(epoch, choice, attestationHash)` referencing a BONAFIDE Fides attestation, and Gensyn AXL handles the off-chain peer coordination (the agents use AXL message bus for vote distribution, then commit the aggregated tally on-chain). The mindX Soul/Mind/Hands cognition stack is the runtime each Soldier operates: Soul = persistent character (sealed in 0G Storage as INFT metadata), Mind = ToolLoopAgent reasoning loop (AI SDK v6), Hands = on-chain action set (the smart-contract callable surface).

The dual-chain bridge is openBDK: every WARCOUNCIL verdict on EVM emits a `VerdictResolved` event that an openBDK relayer mirrors to Algorand via a `application call` to the BOARDROOM stateful contract, where it is recorded as a global state mutation under the

key `verdict:epoch`. Conversely, charter amendments on Algorand (requiring 5-of-7 Pera multisig of Soldier addresses) emit Algorand events that the relayer mirrors back to EVM by calling `DAIOAgentGovernor.updateCharter(bytes32 newHash, bytes calldata stateProof)`, where the state proof is verified via Algorand State Proofs (quantum-resistant Falcon signatures aggregated by validators).

Recommendations

1. **Immediately (Week 1):** Fork `github.com/0gfoundation/0g-agent-nft` at the `eip-7857-draft` branch into the BANKON monorepo and adapt `AgentNFT.sol` into `mindx_agent_inft.sol` with the BONAFIDE hooks shown above. Run the existing Hardhat tests, then port to Foundry. Deploy a dry-run to Galileo testnet (chain ID 16601) using `forge script` against `https://evmrpc-testnet.0g.ai`. **Trigger to escalate:** if the reference verifier's TEE attestation library is not upgrade-compatible with our planned ZKP fallback, schedule a ZKP verifier sprint before mainnet.
2. **Week 2–3:** Wire Parsec to `@x402-avm/avm` v2.6.1 directly — do not fork. Deploy the `MindXX402Middleware` receipt registry. Stand up a self-hosted GoPlausible facilitator at `parsec.bankon.eth.link/facilitator` to avoid dependency on `x402.org/facilitator` for production traffic. **Trigger:** if hosted facilitator round-trip exceeds 1.5s p95 during load test, fall back to AlgoVoi or x402-saas facilitators listed in `awesome-x402`.
3. **Week 4:** Deploy `DAIOAgentGovernor` to 0G Aristotle mainnet with the 13 assessor addresses pre-seeded and the boardroom charter hash committed. Mint the NFD V3 records (`boardroom.bankon.algo`, `warcouncil.bankon.algo`, one per CEO/Soldier role) via `@txnlab/nfd-sdk`. Run a CONCLAVE-1 dry-run with synthetic assessor signatures to validate the ≥10/≥8/5–7/<5 thresholds. **Trigger:** if a single-chapter assessor key is compromised mid-epoch, the recovery path is to invoke `DAIOAgentGovernor.rotateAssessor(uint8 chapter, address newAddr)` requiring 5-of-7 Soldier multisig.
4. **Week 5–6:** Deploy AgenticPlace listing flow with the paired INFT + ARC-19 minting pattern. Index `Listed` events into a Postgres + Drizzle catalog. Stand up mindX provisioning daemon listening to `Minted` events on Aristotle. **Trigger:** mainnet launch criteria are (a) ≥3 successful INFT transfers with valid TEE proofs on Galileo, (b) ≥100 successful x402 settlements on Algorand mainnet against the BANKON facilitator, (c) one full CONCLAVE epoch resolved end-to-end with mainnet assessors.
5. **Ongoing:** Keep the AllChain CAIP-2 loader pinned to verified chain IDs only. Re-verify Galileo chain ID (16601 vs 16602 docs drift) on every release. Subscribe to `eips.ethereum.org/EIPS/eip-7857` for spec changes — when ERC-7857 moves from Draft to Last Call, freeze the BANKON interface IDs and publish a compatibility statement.

Caveats

- **EIP-7857 status:** As of May 2026 the EIP is still in **Draft** status (PR #824 opened January 3, 2025) and not yet Final. The on-chain `iTransfer`/`iClone` selectors and the `TransferValidityProof` struct layout MAY change before finalisation. The 0G Foundation reference at `github.com/0gfoundation/0g-agent-nft` branch `eip-7857-draft` is the authoritative implementation today, but pin commit hashes when forking.
- **Galileo chain ID drift:** The official 0G testnet documentation page at `docs.0g.ai/developer-hub/testnet/testnet-overview` currently shows chain ID 16602 in some Hardhat snippets, while thirdweb, ChainList, and the official "Testnet V3 Galileo relaunch" announcement all use 16601. The 16601 value is correct as of May 2026 — verify before every deployment.
- **0G mainnet `DAEntrance` address:** Not currently listed on the mainnet overview page; only the Galileo testnet page lists `0xE75A073dA5bb7b0eC622170Fd268f35E675a957B`. DA appears to be served through the Flow contract on Aristotle v1.0.4; confirm with 0G Foundation engineering before integrating DA-specific paths in production.
- **ZKP verifier:** The 0G reference Verifier has a `// TODO: add ZKP verification` placeholder in the `verfifyOwnershipProof` function. Production deployments today rely on the TEE path only; ZKP attestations from systems like Risc0 or SP1 are not yet wired in.
- **CAIP-2 strict compliance:** GoPlausible's Algorand CAIP-2 identifiers include `/`, `+`, and `=` padding characters that technically violate the strict CAIP-2 reference grammar (which mandates `[-_a-zA-Z0-9]{1,32}`). This is the de-facto industry usage (WalletConnect, Pera, the Algorand Foundation x402 documentation all use the same form), but consumers parsing identifiers with strict CAIP-2 regexes will need to accommodate the extended character set. Chainagnostic
- **Parsec branding caveat:** No public repository named `parsec-wallet` was located in the GoPlausible org or under a BANKON-owned org; Parsec is a label applied to a BANKON-internal wrapper over `@x402-avm/*`. Treat any external `parsec-wallet` GitHub references as candidates for collision review before publishing under the same name.
- **`MindXAgentINFT.iTransfer` reference snippet:** The code shown above is an authoritative interface skeleton derived directly from EIP-7857's reference Verifier and the 0G Foundation `AgentNFT.sol` pattern. It is not yet audited; the production version requires (at minimum) a reentrancy guard on `iTransfer`/`iClone`, a separate registry for `delegateAccess`, and Slither + Mythril + Echidna runs before any mainnet broadcast.
- **AgenticPlace + ARC-19 pairing:** This document specifies a "shadow" Algorand ASA per INFT for discoverability. This adds operational complexity (two mints, two state machines to keep in sync) and should be re-evaluated once the Algorand x402 marketplace ecosystem matures or NFD V3 directly indexes EVM INFTs.